

Iterative Statements

In Python, looping statements are used to repeatedly execute a block of code. The two main types of loops are "for" and "while" loops.

range()

the `range()` function is used to generate a sequence of numbers, which is often used for iterating over a sequence of numbers in a loop. The general syntax of the `range()` function is:

```
Syntax:  
range(stop)  
range(start, stop)  
range(start, stop, step)
```

1. For Loop:

The `for` loop is used to iterate over a sequence (that is either a list, tuple, dictionary, string, or other iterable objects) and execute a block of code for each element.

Syntax:

```
for variable in sequence:  
    # code to be executed
```

Example:

```
fruits = ['apple', 'banana', 'cherry']  
for fruit in fruits:  
    print(fruit)
```

2. While Loop:

The `while` loop is used to repeatedly execute a block of code as long as a condition is true.

Syntax:

```
while condition:
    # code to be executed
```

Example:

```
count = 0
while count < 3:
    print(count)
    count += 1
```

3. Nested For Loop:

You can have a loop inside another loop. This is called a nested loop.

Example:

```
for i in range(3):

    for j in range(2):
        print(i, j)
```

4. Nested While Loop:

Similarly, you can have a while loop inside another while loop.

Example:

```
Syntax:
while outer_condition:
    # outer loop code
    while inner_condition:
        # inner loop code

Example:
outer = 0
while outer < 3:
    inner = 0
    while inner < 2:
        print(outer, inner)
```

```
        inner += 1  
    outer += 1
```

Looping statements are powerful tools in programming and are used to automate repetitive tasks. The indentation in Python is crucial for defining the block of code inside the loop. The indentation is what makes the code readable and determines which statements belong to the loop.